

COLLEGE CODE : 9623

COLLEGE NAME : AMRITA COLLEGE OF ENGINEERING AND
TECHNOLOGY

DEPARTMENT : CSE

STUDENT NM-ID : 87E5B415F878094BA354428477E60A1F

ROLL NO : 52

DATE : 10/9/2025

COMPLETED THE PROJECT NAMED AS PHASE 2

TECHNOLOGY PROJECT NAME : USER REGISTRATION WITH VALIDATION

SUBMITTED BY,

NAME : C KOKILA

MOBILE NO : 8778200351

1. Tech Stack Selection

Frontend

- Framework: React (or Angular/Vue depending on team skills).
- UI Library: Tailwind CSS or Bootstrap for responsive styling.
- Form Validation: React Hook Form + Yup/Zod for schema-based validation.
- State Management: Redux Toolkit (for larger apps) or Context API (for MVP).

Backend

- Language/Framework: Node.js with Express (lightweight, scalable) OR Django/Flask (Python).
- Validation: Middleware with Joi / Zod / class-validator
- Authentication: JWT (JSON Web Tokens) for session handling.
- Password Security: Hashing with bcrypt/Argon

Database

- Relational: PostgreSQL/MySQL (ACID, constraints).
- ORM/ODM: Sequelize / Prisma (Node.js), Django ORM (Python).

DevOps & Testing

- Deployment: Docker + CI/CD pipeline (GitHub Actions, GitLab CI).
- Testing: Jest/Supertest for backend, Cypress/Playwright for frontend.

2. UI Structure & API Scheme Design

UI Structure

Registration Page:

- Full Name [text]
- Username [text, min 3, unique]
- Email [email format, unique]
- Password [min 8, includes special char, digit, upper/lowercase]
- Confirm Password [match Password]
- Register Button

Error Messages: Inline and user-friendly.

Success Screen: "Registration Successful. Please log in."

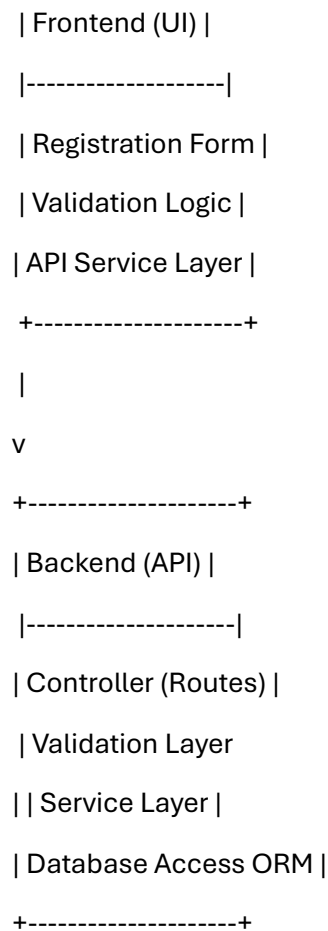
API Scheme

- POST /api/register → Create new user.
- POST /api/login → Authenticate user.
- GET /api/users/:id → Fetch profile (secured).
- POST /api/logout → End session.

3. Data Handling Approach

- Frontend: Validate inputs before submission (prevent bad requests).
- Backend: Re-validate all inputs to ensure security (never trust client).
- Password Storage: Use bcrypt/Argon2 hashing with salt.
- Duplicate Handling: Database constraints (unique email/username).
- Error Handling: Standard JSON responses with error codes.
- Data Privacy: Use HTTPS, secure cookies or JWT.
- Logs: Store minimal info, mask sensitive fields.

4. Component / Module Diagram



1

V

$$+ \text{-----} +$$

| Database (DB) |

| Users Table |

$|-id|$

| - name |

| - username |

| - email |

```
| - password_hash |
```

| - created_at |

$$+ \text{-----} +$$

5. Basic Flow Diagram

[User]

1

V

[Fill Registration Form] | v [Client-side Validation] --invalid--> [Show Errors]

1

valid

V

[Send